SYED ABDUL ALI SHAH · DSA PREP

NeetCode 150

Competitive Programming Guide — Roman Urdu mein step-by-step explanation, complexity analysis, aur pattern recognition. Arrays & Hashing se leke Binary Search tak, 30 core problems.

Arrays & Hashing · Heap · Two Pointers · Sliding Window · Stack · Backtracking · Binary Search · C++ Solutions

---

PART 1

Arrays, Hashing & Heap

Problems 1–6: Contains Duplicate se Product Except Self tak. Hash-based patterns, frequency arrays, aur CPU-level optimization insights — sab Roman Urdu mein.

PROBLEM 1 · ARRAYS AND HASHING

ARRAYS AND HASHING

### 217 — Contains Duplicate

Problem Samajhte Hain

Ek integer array diya hai — check karna hai ke koi element array mein ek se zyada baar aata hai ya nahi.

> **PROBLEM STATEMENT**
> Input: int[] nums   Output: true - agar koi duplicate ho, false - agar sab unique hon
> Example: [1,2,3,1] → true | [1,2,3,4] → false

Core Intuition — HashSet Existence Check

"Ye element pehle dekha hai kya?" — sirf existence check karni hai, frequency nahi. Array traverse karo, har element ko ek unordered_set mein daalte jao. Insert se pehle check karo ke element pehle se set mein hai ya nahi — agar hai, duplicate mil gaya.

```cpp
class Solution {
public:
    bool containsDuplicate(vector<int>& nums) {
        unordered_set<int> seenElements;
        seenElements.reserve(nums.size()); // Rehashing avoid karo
        for (int num : nums) {
            if (seenElements.count(num)) {
                return true; // Duplicate mila!
            }
            seenElements.insert(num);
        }
        return false;
    }
};
// Time: O(n) average | Space: O(n)
```

> **INTERVIEW INSIGHT**
> unordered_set > set yahan — set (ordered) internally BST hai (O(log n) per op), unordered_set hash table hai (O(1) average). reserve(n) pehle se memory le leta hai, rehashing avoid hoti hai.

> **YAAD RAKHNE WALI CHEEZEIN**
> - Pattern: HashSet Existence Check — O(1) lookup
> - set → O(n log n) | unordered_set → O(n) average
> - reserve(n) se rehashing avoid hoti hai
> - Ye wahi pattern hai jo Longest Consecutive Sequence (#128) aur Valid Sudoku (#36) mein reuse hota hai

PROBLEM 2 · ARRAYS AND HASHING

ARRAYS AND HASHING

## 242 — Valid Anagram

Problem Samajhte Hain

Do strings s aur t di hoti hain. Check karna hai ke dono anagram hain ya nahi — matlab ek string ke saare letters dusri mein bhi same frequency ke saath maujood hon.

> **PROBLEM STATEMENT**
> Input: string s, string t (sirf lowercase a-z)   Output: true - agar t, s ka anagram ho
> Example: s="anagram", t="nagaram" → true

Core Intuition — Frequency Array

"Kitni baar aaya hai?" — count matter karta hai, sirf presence nahi. Alphabet fixed (26 letters) hai, isliye hash map ki zaroorat nahi — ek int arr[26] se dono strings ki frequency ek hi loop mein track ho sakti hai: s ke liye increment, t ke liye decrement. Agar anagram hai toh final mein sab zero.

```
class Solution {
public:
    bool isAnagram(string s, string t) {
        if (s.length() != t.length()) return false;
        int alphabetsCount[26] = {0};
        for (int i = 0; i < s.length(); i++) {
            alphabetsCount[s.at(i) - 'a']++;
            alphabetsCount[t.at(i) - 'a']--;
        }
        for (int i = 0; i < 26; i++) {
            if (alphabetsCount[i] != 0) return false;
        }
        return true;
    }
};
// Time: O(n) | Space: O(1) — 26 fixed slots
```

> **COMMON MISTAKE**
> unordered_map<int,int> use karna aur us par insert() se count update karne ki koshish — insert() existing key ko update NAHI karta. Fixed alphabet ho toh hamesha int arr[26] use karo.

> **YAAD RAKHNE WALI CHEEZEIN**
> - Pattern: Frequency Array — counting, not just existence
> - s[i]-'a' trick: char ko 0-25 index mein convert karta hai
> - Yahi frequency-array trick Group Anagrams (#49) aur Permutation in String (#567) mein reuse hota hai

PROBLEM 3 · ARRAYS AND HASHING

ARRAYS AND HASHING

## 1 — Two Sum

Problem Samajhte Hain

Ek array aur target diya hai. Dhundna hai kaunse do indices ke elements ka sum target ke barabar hai. Exactly ek solution guaranteed hai.

> **PROBLEM STATEMENT**
> Input: vector<int> nums, int target   Output: vector<int> — do indices jinka sum == target
> Example: nums=[2,7,11,15], target=9 → [0,1]

Core Intuition — Hash Map + Complement

Har element ke liye complement (target - nums[i]) dhundo. Map ko gradually build karo — ek hi pass mein: pehle complement check karo, agar mile toh return, warna current element insert karo.

```
class Solution {
public:
    vector<int> twoSum(vector<int>& nums, int target) {
        unordered_map<int, int> elements;
        for (int i = 0; i < nums.size(); i++) {
            int k = target - nums[i]; // Complement
            if (elements.count(k)) {
                return {elements[k], i};
```

```
            }
            elements.insert({nums[i], i});
        }
        return {};
    }
};
// Time: O(n) average | Space: O(n)
```

INTERVIEW INSIGHT

count() > at(): at() key na milne par exception throw karta hai jo CPU ke liye heavy hai. 1-pass gradual map build karna 2-pass se behtar hai — early exit possible hota hai.

YAAD RAKHNE WALI CHEEZEIN

- Pattern: Hash Map + Complement
- complement = target - nums[i] — hash problems mein common trick
- Sorted array wala variant Two Sum II (#167) two-pointers se O(1) space mein solve hota hai

PROBLEM 4 · ARRAYS AND HASHING

ARRAYS AND HASHING

## 49 — Group Anagrams

Problem Samajhte Hain

Strings ka array diya hai — anagram strings ko ek group mein rakhna hai aur sab groups return karne hain.

PROBLEM STATEMENT

Input: vector<string> strs   Output: vector<vector<string>> — anagram groups

Example: ["eat","tea","tan","ate","nat","bat"] → [["bat"],["nat","tan"],["ate","eat","tea"]]

Core Intuition — Frequency Array as Map Key

Har string ka ek unique "code" banao — 26-letter frequency array se, '#' separator ke saath. Jo strings anagram hain unka code same hoga, isliye ye code hashmap ki key ban jata hai. Ye Valid Anagram (#242) ka wahi frequency-array trick hai, bas ab key-generation ke liye use ho raha hai.

```cpp
class Solution {
private:
    string getCode(string str) {
        int alphabets[26] = {0};
        for (char c : str) { alphabets[c - 'a']++; }
        string code = "";
        for (int i : alphabets) {
            code += i + 'a';
            code += '#'; // Separator (important!)
        }
        return code;
    }
public:
    vector<vector<string>> groupAnagrams(vector<string>& strs) {
        unordered_map<string, vector<string>> groupingMap;
        for (string str : strs) {
            groupingMap[getCode(str)].push_back(str);
        }
        vector<vector<string>> anagramsGroups;
        for (auto& it : groupingMap) {
            anagramsGroups.push_back(it.second);
        }
        return anagramsGroups;
    }
};
// Time: O(n*k) | Space: O(n*k) (k = max string length)
```

COMMON MISTAKE

Key banane ke liye sort(str) use karna — ye O(k log k) per string leta hai. Frequency-array key sirf O(k) leti hai, isliye hamesha better hai. '#' separator zaroori hai warna values ambiguous ho sakti hain (a=1,b=11 vs a=11,b=1).

YAAD RAKHNE WALI CHEEZEIN

- Pattern: Frequency Array (as hashmap key)
- Frequency key O(k) > Sort key O(k log k)
- Anagram grouping ka key: jo same anagrams ke liye same ho

PROBLEM 5 · HEAP

HEAP

347 — Top K Frequent Elements

Problem Samajhte Hain

Array aur number k diya hai. Woh k elements return karne hain jo array mein sabse zyada baar aaye hain.

> **PROBLEM STATEMENT**
> Input: vector<int> nums, int k   Output: vector<int> — top k most frequent elements
> Example: nums=[1,1,1,2,2,3], k=2 → [1,2]

Core Intuition — Frequency + Min-Heap

Pehle frequency table banao (freq[num]++). Phir size-k ka min-heap maintain karo jisme {frequency, number} pairs store hon. Jab heap ka size k se bada ho jaye, sabse kam frequent element (top) pop kar do. End mein heap mein exactly k most frequent elements bachte hain.

```
class Solution {
public:
    vector<int> topKFrequent(vector<int>& nums, int k) {
        if (k == nums.size()) return nums;
        unordered_map<int, int> frequencyTable;
        for (int num : nums) {
            frequencyTable[num]++;
        }
        priority_queue<
            pair<int, int>,
            vector<pair<int, int>>,
            greater<pair<int, int>>
        > heap;
        for (auto item : frequencyTable) {
            heap.push({item.second, item.first}); // {freq, num}
            if (heap.size() > k) {
                heap.pop(); // Sabse kam frequent nikalo
            }
        }
        vector<int> ans(k, 0);
        for (int i = 0; i < k; i++) {
            ans[i] = heap.top().second; // Pehle top()
            heap.pop(); // phir pop()
        }
        return ans;
    }
};
// Time: O(n + m log k) | Space: O(n) (m = unique elements)
```

> **COMMON MISTAKE**
> heap.pop() void return karta hai — value nikalne ke liye pehle .top() lo, phir .pop() karo. Size check ==k ki jagah >k use karo, warna ek extra element pop ho jayega.

> **YAAD RAKHNE WALI CHEEZEIN**
> - Pattern: Frequency Array/Map + Min-Heap
> - Min-heap: greater<T> comparator — chota element top par
> - Heap mein {freq, element} store karo — freq se ordering hogi
> - Size >k check: ==nahi, > use karo

PROBLEM 6 · ARRAYS

ARRAYS

238 — Product of Array Except Self

Problem Samajhte Hain

Har element ke liye baaki sab elements ka product return karna hai — division use kiye baghair, O(n) time aur O(1) extra space mein.

PROBLEM STATEMENT

Input: vector<int> nums   Output: ans[i] = product of all nums except nums[i]
Example: nums=[1,2,3,4] → ans=[24,12,8,6]
Constraint: division use nahi kar sakte

Core Intuition — Prefix x Postfix

Har position i ke liye: ans[i] = (i ke LEFT ka product) x (i ke RIGHT ka product). Pehle ek left-to-right pass mein prefix products ans array mein store karo, phir ek right-to-left pass mein postfix products se multiply karo.

```
class Solution {
public:
    vector<int> productExceptSelf(vector<int>& nums) {
        vector<int> ans(nums.size(), 1);
        int pre = 1;
        for (int i = 0; i < nums.size(); i++) {
            ans[i] = pre;
            pre *= nums[i];
        }
        int post = 1;
        for (int i = nums.size() - 1; i >= 0; i--) {
            ans[i] *= post;
            post *= nums[i];
        }
        return ans;
    }
};
// Time: O(n) | Extra Space: O(1)
```

INTERVIEW INSIGHT

Division approach zero elements par fail hoti hai. Prefix x Postfix hamesha safe hai. Order critical hai: pehle ans[i]=pre store karo, PHIR pre*=nums[i] karo — warna current element khud include ho jayega.

YAAD RAKHNE WALI CHEEZEIN

- Pattern: 2-Pass Prefix x Postfix (left-to-right, phir right-to-left)
- O(1) extra space: sirf 2 running variables (pre, post)
- Division ki zaroorat nahi — zero-safe approach

Part 1 Ka Pattern Summary

Part 1 ke 6 problems teen patterns mein aate hain — yahi patterns aage Two Pointers, Sliding Window, Stack, aur Backtracking ke saath mil kar poora 30-problem map banate hain.

PROBLEM	PATTERN	TIME	SPACE
#217 Contains Duplicate	HashSet Existence Check	O(n)	O(n)
#242 Valid Anagram	Frequency Array	O(n)	O(1)
#1 Two Sum	Hash Map + Complement	O(n)	O(n)
#49 Group Anagrams	Frequency Array (as key)	O(nk)	O(nk)
#347 Top K Frequent	Frequency + Min-Heap	O(n log k)	O(n)
#238 Product Except Self	Prefix x Postfix	O(n)	O(1)

DEKHO KAISE YE CHAIN BANTI HAI

Contains Duplicate (#217) ka "existence check" pattern seedha Longest Consecutive Sequence aur Valid Sudoku (Part 2) mein reuse hota hai. Valid Anagram (#242) ka frequency-array trick Group Anagrams mein key ke roop mein, aur Part 2 mein sliding window ke andar (Permutation in String, Longest Repeating Char Replacement) dubara istemal hota hai.

PART 2

Two Pointers & Sliding Window

Problems 7–19: Valid Sudoku se Sliding Window Maximum tak. Converging pointers, expand-shrink windows, aur monotonic deque.

PROBLEM 7 · ARRAYS AND HASHING

ARRAYS AND HASHING

## 36 — Valid Sudoku

Problem Samajhte Hain

Valid Sudoku mein ek 9x9 Sudoku board diya hota hai (kuch cells filled, kuch '.' se empty). Check karna hota hai ke jo digits already board par likhi hain woh valid Sudoku rules follow karti hain ya nahi. Poora board solve nahi karna — sirf validity check karni hai.

> **PROBLEM STATEMENT**
> Rules: Har row/column/3x3 sub-box mein 1-9 digit sirf ek baar aa sakti hai

Core Intuition — Teen HashSet Ek Saath

Idea simple hai: har row, har column, aur har 3x3 box ke liye alag unordered_set maintain karo. Jaise hi board par ek cell traverse karo, teeno relevant sets mein check karo ke ye digit pehle se maujood toh nahi. Agar hai — invalid. Warna teeno sets mein insert kar do aur aage badho.

```
class Solution {
public:
    bool isValidSudoku(vector<vector<char>>& board) {
        int N = 9;
        vector<unordered_set<char>> rowSet(N);
        vector<unordered_set<char>> colSet(N);
        vector<unordered_set<char>> boxSet(N);
        for (int r = 0; r < board.size(); r++) {
            for (int c = 0; c < board[r].size(); c++) {
                char val = board[r][c];
                if (val == '.') { continue; }
                if (rowSet[r].count(val)) { return false; }
                rowSet[r].insert(val);
                if (colSet[c].count(val)) { return false; }
                colSet[c].insert(val);
                int box = r / 3 * 3 + c / 3;
                if (boxSet[box].count(val)) { return false; }
                boxSet[box].insert(val);
            }
        }
        return true;
    }
};
// Time: O(1) | Space: O(1) — fixed 9x9 board
```

CELL(R,C)	R/3	C/3	BOX = (R/3)*3 + (C/3)
(0,0)	0	0	0
(0,5)	0	1	1
(4,4)	1	1	4
(8,8)	2	2	8

> **COMMON MISTAKE**
> Sirf row aur column check karke box bhool jana — ya box index formula galat likhna (jaise r*3+c ya r/3+c/3 bina multiply kiye). Dono se galat result aata hai.

YAAD RAKHNE WALI CHEEZEIN

- Fixed 9x9 size ki wajah se ye problem O(1) time/space hai
- Teen sets simultaneously maintain karo: row, column, box
- Box formula: (r/3)*3 + (c/3) — integer division important hai
- Solve nahi karna, sirf validate karna hai

PROBLEM 8 · ARRAYS AND HASHING

ARRAYS AND HASHING

## 128 — Longest Consecutive Sequence

Problem Samajhte Hain

Ek unsorted integer array diya hai. Dhundna hai sabse lambi consecutive sequence ki length — matlab consecutive integers jo array mein maujood hon, chahe original array mein woh kahin bhi scattered hon.

> **PROBLEM STATEMENT**
> Input: vector<int> nums   Output: int — longest consecutive elements sequence ki length
> Example: nums=[100,4,200,1,3,2] → 4 (sequence: 1,2,3,4)
> Target: O(n) time — sorting allowed nahi

Core Intuition — Sirf Sequence Start Se Shuru Karo

Sabse important trick: har number ke liye poori sequence mat expand karo — sirf un numbers se shuru karo jo kisi sequence ke start hain. Ek number n sequence ka start hai agar (n-1) set mein nahi hai.

```
class Solution {
public:
    int longestConsecutive(vector<int>& nums) {
        unordered_set<int> numsSet;
        for (int n : nums) { numsSet.insert(n); }
        int currentSequenceLength = 0;
        int maxSequenceLength = 0;
        for (int n : numsSet) {
            if (numsSet.count(n - 1)) { continue; // beech ka number hai, skip }
            else {
                currentSequenceLength++;
                int v = n;
                while (numsSet.count(v + 1)) {
                    currentSequenceLength += 1;
                    v++;
                }
                maxSequenceLength = max(maxSequenceLength, currentSequenceLength);
                currentSequenceLength = 0;
            }
        }
        return maxSequenceLength;
    }
};
// Time: O(n) amortized
```

> **Kyun O(n) hai, O(n²) nahi?**
> Nested loop (for + while) dikh raha hai, lekin inner while loop sirf sequence-start numbers par chalta hai — har number life mein sirf ek baar visit hota hai. Total amortized O(n).

> **YAAD RAKHNE WALI CHEEZEIN**
> - Sirf sequence-starts se shuru karo: check karo (n-1) set mein hai ya nahi
> - unordered_set se O(1) existence check — sorting ki zaroorat nahi
> - Nested loop dikhne ke bawajood total complexity O(n) hai
> - Set based approach sorting approach (O(n log n)) se better hai

PROBLEM 9 · TWO POINTERS

TWO POINTERS

## 125 — Valid Palindrome

Problem Samajhte Hain

Ek string di hai jisme letters, digits, spaces, aur punctuation ho sakti hai. Check karna hai ke ye string palindrome hai ya nahi — sirf alphanumeric characters count karte hue, case-insensitive tareeqe se.

> **PROBLEM STATEMENT**
> Input: string s   Output: true/false — alphanumeric-only, lowercase-normalized palindrome check
> Example: "A man, a plan, a canal: Panama" → true

Core Intuition — Two Pointers Converge

Ek pointer left se shuru, ek right se. Dono ek dusre ki taraf move karte hain. Har step par: non-alphanumeric characters skip karo, phir lowercase compare karo.

```
class Solution {
public:
    bool isPalindrome(string s) {
        int left = 0, right = s.size() - 1;
        while (left < right) {
            while (left < right && !isalnum(s.at(left))) { left++; }
            while (right > left && !isalnum(s.at(right))) { right--; }
            if (tolower(s.at(left)) != tolower(s.at(right))) {
                return false;
```

```
            }
            left++; right--;
        }
        return true;
    }
};
// Time: O(n) | Space: O(1)
```

COMMON MISTAKE
Naya cleaned string bana kar phir reverse karke compare karna — ye O(n) extra space use karta hai. Two-pointer approach O(1) space mein same kaam karta hai.

YAAD RAKHNE WALI CHEEZEIN

- Two pointers converge pattern: left aur right dono middle ki taraf move
- isalnum() + tolower(): dono standard library functions interview mein useful
- O(1) space — naya string banane ki zaroorat nahi
- Guard conditions (left<right) inner skip-loops mein bhoolna common bug hai

PROBLEM 10 · TWO POINTERS

TWO POINTERS

167 — Two Sum II (Sorted Array)

Problem Samajhte Hain

Ye Two Sum ka variant hai — lekin array pehle se sorted hai (ascending order). Is wajah se hash map ki zaroorat nahi — two pointers se O(n) time aur O(1) space mein solve ho sakta hai.

PROBLEM STATEMENT
Input: vector<int> numbers (1-indexed, sorted), int target   Output: vector<int> — 1-indexed do indices jinka sum == target
Example: numbers=[2,7,11,15], target=9 → [1,2]

Core Intuition — Sorted Array Ka Faida Uthao

Chunki array sorted hai: agar current sum target se bada hai, right pointer ko left move karo. Agar sum chota hai, left pointer ko right move karo. Ye monotonic property hi two pointers ko kaam karne deti hai.

```
class Solution {
public:
    vector<int> twoSum(vector<int>& numbers, int target) {
        int left = 0;
        int right = numbers.size() - 1;
        while (left < right) {
            if (numbers[left] + numbers[right] > target) { right--; continue; }
            else if (numbers[left] + numbers[right] < target) { left++; continue; }
            else { return {left + 1, right + 1}; }
        }
        return {};
    }
};
// Time: O(n) | Space: O(1)
```

INTERVIEW INSIGHT
Jab bhi input "sorted" mile — turant socho ke kya two pointers se O(1) space mein solve ho sakta hai. Sorted property ek strong hint hoti hai.

YAAD RAKHNE WALI CHEEZEIN

- Sorted array + pair-sum problem = Two Pointers ka classic signal
- sum > target: right-- | sum < target: left++
- 1-indexed return: left+1, right+1 yaad rakho
- Hash map se better: O(1) space jab array already sorted ho

PROBLEM 11 · TWO POINTERS

TWO POINTERS

## 15 — 3Sum

Problem Samajhte Hain

Array mein teen numbers dhundo jinka sum zero ho — aur unique triplets return karo (duplicates nahi).

> **PROBLEM STATEMENT**
> Input: vector<int> nums   Output: vector<vector<int>> — saare unique triplets [a,b,c] jinka a+b+c=0
> Example: nums=[-1,0,1,2,-1,-4] → [[-1,-1,2],[-1,0,1]]

Core Intuition — Sort + Fix + Two Pointers

Pehle array sort karo. Phir har index i ko "fixed" number treat karo, aur baaki array mein two-pointer se dhundo jinka sum == -nums[i]. Duplicates skip karne ke liye dono jagah careful checks lagane padte hain.

```
class Solution {
public:
    vector<vector<int>> threeSum(vector<int>& nums) {
        sort(nums.begin(), nums.end());
        vector<vector<int>> resultList;
        for (int i = 0; i < nums.size(); i++) {
            if (nums[i] > 0) { break; }
            if (i > 0 && nums[i] == nums[i - 1]) { continue; }
            int left = i + 1;
            int right = nums.size() - 1;
            while (left < right) {
                int sum = nums[i] + nums[left] + nums[right];
                if (sum < 0) { left++; }
                else if (sum > 0) { right--; }
                else {
                    resultList.push_back({nums[i], nums[left], nums[right]});
                    left++; right--;
                    while (left < right && nums[left] == nums[left - 1]) { left++; }
                    while (left < right && nums[right] == nums[right + 1]) { right--; }
                }
            }
        }
        return resultList;
    }
};
// Time: O(n²) | Space: O(1) extra
```

> **COMMON MISTAKE**
> Duplicate skip sirf outer loop mein karna aur inner left/right pointers mein bhool jana — is se duplicate triplets result mein aa jate hain.

YAAD RAKHNE WALI CHEEZEIN

- 3Sum = Sort + Fix one number + Two Pointers on rest
- nums[i]>0 par break: sorted array mein useful early exit
- Duplicates 3 jagah skip hote hain: outer i, inner left, inner right
- Time $O(n^2)$ — brute force $O(n^3)$ se bahut behtar

PROBLEM 12 · TWO POINTERS

TWO POINTERS

## 11 — Container With Most Water

Problem Samajhte Hain

Heights ka ek array hai — har index ek vertical line represent karta hai. Do lines choose karni hain jo container banayein jisme sabse zyada pani store ho.

> **PROBLEM STATEMENT**
> Input: vector<int> height   Output: int — maximum water area
> Example: height=[1,8,6,2,5,4,8,3,7] → 49

Core Intuition — Greedy Two Pointers

Left (0) aur right (last) se shuru karo. Har step par, jo line CHOTI hai usse move karo — kyunki wahi bottleneck hoti hai. Badi line ko move karne se area kabhi barabar ya kam hi hoga, zyada nahi.

```
class Solution {
public:
    int maxArea(vector<int>& height) {
        int left = 0;
        int right = height.size() - 1;
        int maxArea = 0;
        while (left < right) {
```

```
            int width = right - left;
            int cheight = min(height[left], height[right]);
            maxArea = max(maxArea, width * cheight);
            if (height[left] > height[right]) { right--; }
            else { left++; }
        }
        return maxArea;
    }
};
// Time: O(n) | Space: O(1)
```

> **Kyun choti line move karna sahi hai?**
> Agar left line choti hai aur hum right ko move karein, naya width kam ho jayega AUR height ab bhi purani (choti) height se bounded rahegi — area guaranteed kam ya barabar hoga, kabhi zyada nahi.

> **YAAD RAKHNE WALI CHEEZEIN**
> - Area = width x min(height[left], height[right])
> - Greedy move: hamesha CHOTI line wala pointer move karo
> - Widest container se shuru karo (index 0 aur n-1)
> - O(n) vs brute-force $O(n^2)$ — classic two-pointer optimization

PROBLEM 13 · TWO POINTERS

TWO POINTERS

## 42 — Trapping Rain Water

Problem Samajhte Hain

Elevation map di hai — har bar ki height. Calculate karna hai ke baarish ke baad kitna pani trap hoga bars ke beech.

> **PROBLEM STATEMENT**
> Input: vector<int> height   Output: int — total trapped rainwater units
> Example: height=[0,1,0,2,1,0,1,3,2,1,2,1] → 6
> Key idea: har position par pani = min(leftMax, rightMax) - height[i]

Core Intuition — Har Position Ka Water Level

Kisi bhi index i par kitna pani ruk sakta hai? Ye depend karta hai us index ke LEFT ki max height aur RIGHT ki max height mein se jo CHOTA ho. Two-pointer trick isse O(n) mein le aata hai.

```
class Solution {
public:
    int trap(vector<int>& height) {
        int left = 0, right = height.size() - 1, total = 0;
        int leftMax = height[left], rightMax = height[right];
        while (left < right) {
            if (height[left] < height[right]) {
                leftMax = max(leftMax, height[left]);
                if (leftMax - height[left] > 0) { total += leftMax - height[left]; }
                left++;
            } else {
                rightMax = max(rightMax, height[right]);
                if (rightMax - height[right] > 0) { total += rightMax - height[right]; }
                right--;
            }
        }
        return total;
    }
};
// Time: O(n) | Space: O(1)
```

APPROACH	TIME	SPACE	IDEA
Brute Force	$O(n^2)$	O(1)	Har index ke liye left/right max dhundo
DP Precompute	O(n)	O(n)	leftMax[], rightMax[] arrays banao
Two Pointers	O(n)	O(1)	Running max, chota side move karo

YAAD RAKHNE WALI CHEEZEIN

- Water at index i = min(leftMax, rightMax) - height[i]
- Choti height wale pointer ko move karo
- Running max (leftMax/rightMax) O(n) precompute arrays ki jagah use hota hai
- O(n) time O(1) space — optimal, DP se bhi behtar space-wise

PROBLEM 14 · SLIDING WINDOW

SLIDING WINDOW

## 121 — Best Time to Buy and Sell Stock

Problem Samajhte Hain

Prices ka array hai — ek din price par khareedo, baad ke kisi din bech do. Maximum profit dhundo. Sirf ek transaction allowed.

PROBLEM STATEMENT

Input: vector<int> prices   Output: int — maximum achievable profit

Example: prices=[7,1,5,3,6,4] → 5 (buy@1, sell@6)

Core Intuition — One-Pass Sliding Window

"right" pointer array traverse karta hai, aur "buy" variable minimum-so-far track karta hai (implicit left boundary). Har naye price par: agar ye minimum se kam hai, buy update karo. Warna profit calculate karo.

```
class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int profit = 0;
        int buy = prices[0];
        for (int i = 1; i < prices.size(); i++) {
            if (prices[i] < buy) {
                buy = prices[i];
                continue;
            }
            profit = max(profit, prices[i] - buy);
        }
        return profit;
    }
};
// Time: O(n) | Space: O(1)
```

INTERVIEW INSIGHT

Ye problem Kadane's Algorithm se conceptually related hai — "running minimum + running max profit" wahi pattern hai jo max-subarray-sum mein "running sum" ka hota hai.

YAAD RAKHNE WALI CHEEZEIN

- Running minimum (buy) track karo — ye implicit window-left hai
- Har step par profit = current price - running min
- O(n) single pass — brute force O(n²) se best
- Sirf ek transaction — multiple buy/sell wala variant alag problem hai

PROBLEM 15 · SLIDING WINDOW

SLIDING WINDOW

## 3 — Longest Substring Without Repeating Characters

Problem Samajhte Hain

Ek string di hai — sabse lambi substring dhundo jisme koi character repeat na ho. Classic variable-size sliding window.

PROBLEM STATEMENT

Input: string s   Output: int — longest substring length bina repeating characters ke

Example: s="abcabcbb" → 3 ("abc")

Core Intuition — Expand Right, Shrink Left

Right pointer string expand karta hai. Agar current character pehle se window mein maujood hai, toh left pointer ko tab tak aage badhao jab tak duplicate hata na diya jaye.

```
class Solution {
public:
    int lengthOfLongestSubstring(string s) {
        if (s.size() == 0) { return 0; }
        int left = 0, right = 0, ans = 0;
        unordered_set<char> visitedChars;
        while (right < s.size()) {
            char c = s.at(right);
            while (visitedChars.count(c)) {
                visitedChars.erase(s.at(left));
                left++;
            }
            visitedChars.insert(s.at(right));
            ans = max(ans, right - left + 1);
            right++;
        }
        return ans;
    }
};
// Time: O(n) amortized | Space: O(min(m,n))
```

COMMON MISTAKE

while(visitedChars.count(c)) ki jagah if likhna — is se sirf ek element shrink hota hai, jabki multiple duplicates ho sakte hain jo poori tarah remove karna zaroori hai.

YAAD RAKHNE WALI CHEEZEIN

- Variable sliding window: right expand, duplicate mile toh left shrink (while loop se)
- HashSet current-window ke unique characters track karta hai
- Answer hamesha window size (right-left+1) ka running max hai
- Amortized O(n) — dono pointers milke total 2n steps se zyada move nahi karte

PROBLEM 16 · SLIDING WINDOW

SLIDING WINDOW

## 424 — Longest Repeating Character Replacement

Problem Samajhte Hain

Ek string aur ek integer k diya hai. Aap string mein kisi bhi k characters ko kisi bhi doosre character se replace kar sakte ho. Dhundna hai sabse lambi substring ki length jo replacement ke baad saare-same-character ban jaye.

PROBLEM STATEMENT

Input: string s (uppercase A-Z), int k   Output: int — longest possible same-character substring length after ≤k replacements

Example: s="AABABBA", k=1 → 4 ("AABA" → "AAAA")

Core Intuition — Window Valid Rehti Hai Agar Replacements ≤k

Formula: window size - maxFreq (sabse zyada frequent character ka count) = kitne characters replace karne padenge. Agar ye value k se zyada ho jaye, window invalid hai — shrink karo. maxFreq ko kabhi decrease nahi karte kyunki hume sirf window ka MAX SIZE chahiye.

```
class Solution {
public:
    int characterReplacement(string s, int k) {
        int left = 0, maxFreq = 0, ans = 0;
        int freq[26] = {0};
        for (int right = 0; right < s.size(); right++) {
            freq[s[right] - 'A']++;
            maxFreq = max(maxFreq, freq[s[right] - 'A']);
            while ((right - left + 1) - maxFreq > k) {
                freq[s[left] - 'A']--;
                left++;
            }
            ans = max(ans, right - left + 1);
        }
        return ans;
    }
};
// Time: O(n) | Space: O(1) (26 fixed)
```

maxFreq kabhi decrease kyun nahi hota?
Jab window shrink hoti hai, maxFreq ko update nahi karte — stale value reh sakti hai. Ye theek hai kyunki agar kabhi utni badi window mil chuki thi, answer already utna bada set ho chuka hai. Window ab sirf tabhi grow karegi jab naya bada maxFreq milega.

YAAD RAKHNE WALI CHEEZEIN

- Formula: window_size - maxFreq > k matlab window invalid — shrink karo
- maxFreq kabhi decrease nahi hota — window ka historical max hai
- int freq[26] — Valid Anagram jaisa frequency-array pattern, sliding window ke andar
- O(n) time O(1) space — optimal

PROBLEM 17 · SLIDING WINDOW

SLIDING WINDOW

## 567 — Permutation in String

Problem Samajhte Hain

Do strings s1 aur s2 di hain. Check karna hai ke s2 mein s1 ka koi permutation (anagram) as a substring maujood hai ya nahi.

PROBLEM STATEMENT
Input: string s1, string s2   Output: true — agar s2 mein s1 ka koi anagram substring maujood ho
Example: s1="ab", s2="eidbaooo" → true ("ba")

Core Intuition — Fixed-Size Window + Frequency Match

Window size hamesha s1.length() ke barabar rehti hai. Har window position par, uski frequency array s1 ki frequency array se match hoti hai ya nahi check karo. Window slide karne ke liye ek naya character add hota hai aur sabse purana remove — O(1) per slide.

```
class Solution {
public:
    bool checkInclusion(string s1, string s2) {
        if (s1.length() > s2.length()) { return false; }
        int s1Map[26] = {0}, s2Map[26] = {0};
        for (int i = 0; i < s1.length(); i++) {
            s1Map[s1[i] - 'a']++;
            s2Map[s2[i] - 'a']++;
        }
        for (int i = 0; i < s2.length() - s1.length(); i++) {
            if (match(s1Map, s2Map)) { return true; }
            s2Map[s2[i + s1.length()] - 'a']++;
            s2Map[s2[i] - 'a']--;
        }
        return match(s1Map, s2Map);
    }
    bool match(int* s1Map, int* s2Map) {
        for (int i = 0; i < 26; i++) {
            if (s1Map[i] != s2Map[i]) { return false; }
        }
        return true;
    }
};
// Time: O(n) | Space: O(1)
```

COMMON MISTAKE
Har window ke liye poori frequency array dobara build karna — ye O(n*k) bana deta hai. Sliding trick (add naya, remove purana) use karke O(n) rakhna zaroori hai.

YAAD RAKHNE WALI CHEEZEIN

- Fixed-size window = s1.length() — Longest Substring (variable-size) se alag category
- Frequency array match (int arr[26]) — Valid Anagram ka wahi pattern
- Slide mein O(1) update: naya add, purana remove — poori array recompute mat karo
- match() function 26 comparisons — O(1) constant, poore algorithm ko O(n) rakhta hai

PROBLEM 18 · SLIDING WINDOW

SLIDING WINDOW

## 76 — Minimum Window Substring

Problem Samajhte Hain

Ye sliding window ka sabse advanced problem hai is set mein. Do strings s aur t di hain. Sabse CHOTI substring dhundo s mein jisme t ke saare characters (unki frequency ke saath) maujood hon.

> **PROBLEM STATEMENT**
> Input: string s, string t   Output: string — smallest window jisme t ke saare characters (frequency match) maujood hon
> Example: s="ADOBECODEBANC", t="ABC" → "BANC"
> Empty string "" return karo agar koi valid window na ho

Core Intuition — Variable Window + "required vs create" Counter

Do counters use hote hain: required (t mein kitne DISTINCT characters hain) aur create (window mein abhi kitne characters ki frequency t ke barabar ho chuki hai). Jab create == required, window valid hai — ab shrink karke minimum dhundo.

```
class Solution {
public:
    string minWindow(string s, string t) {
        if (s.size() == 0 || t.size() == 0 || s.size() < t.size()) { return ""; }
        unordered_map<char, int> tMap, sMap;
        int l = 0, r = 0;
        for (char c : t) { tMap[c]++; }
        int required = tMap.size();
        int create = 0;
        int ans[3] = {-1, 0, 0};
        while (r < s.size()) {
            sMap[s[r]]++;
            if (tMap.find(s[r]) != tMap.end() && tMap[s[r]] == sMap[s[r]]) {
                create++;
            }
            while (l <= r && required == create) {
                char c = s[l];
                if (ans[0] == -1 || ans[0] >= r - l + 1) {
                    ans[0] = r - l + 1; ans[1] = l; ans[2] = r;
                }
                sMap[c]--;
                if (tMap.find(c) != tMap.end() && tMap[c] > sMap[c]) {
                    create--;
                }
                l++;
            }
            r++;
        }
        if (ans[0] == -1) { return ""; }
        return s.substr(ans[1], ans[0]);
    }
};
// Time: O(n+m) | Space: O(m)
```

> **INTERVIEW INSIGHT**
> Minimum Window Substring "hardest sliding window" LeetCode par consider hota hai — required/create counter technique isse Longest Substring Without Repeat se distinguish karti hai: yahan FREQUENCY match chahiye, sirf uniqueness nahi.

YAAD RAKHNE WALI CHEEZEIN

- required = t ke distinct characters | create = window mein match ho chuke characters
- create==required: window valid — ab MINIMIZE karne ke liye shrink karo
- Ye variable window hai jisme expand AND shrink dono active roles play karte hain
- ans[0]==-1 check zaroori hai "no valid window found" case ke liye
- Pattern: Longest Substring Without Repeat (max) ka "mirror" hai Minimum Window Substring (min)

PROBLEM 19 · SLIDING WINDOW

SLIDING WINDOW

## 239 — Sliding Window Maximum

Problem Samajhte Hain

Ek array aur window size k diya hai. Har k-size window ke liye maximum element dhundo, jaise window array ke upar left se right slide karti hai.

PROBLEM STATEMENT

Input: vector<int> nums, int k   Output: vector<int> — har window ka maximum

Example: nums=[1,3,-1,-3,5,3,6,7], k=3 → [3,3,5,5,6,7]

Core Intuition — Monotonic Decreasing Deque

Ek deque mein INDICES store karo (values nahi), aur deque ko hamesha monotonic decreasing rakho — matlab front se back tak values kam hoti jayein. Naya element aane par, back se saare CHOTE elements pop kar do (wo kabhi max nahi ban sakte jab bade wale window mein maujood hain). Front automatically hamesha current window ka maximum hota hai. Agar front index window se bahar nikal jaye (expired), usse bhi pop kar do.

```
class Solution {
public:
    vector<int> maxSlidingWindow(vector<int>& nums, int k) {
        vector<int> result(nums.size() - k + 1);
        deque<int> indicesQueue;
        for (int i = 0; i < nums.size(); i++) {
            while (indicesQueue.size() != 0 && indicesQueue.front() < i - k + 1) {
                indicesQueue.pop_front();
            }
            while (indicesQueue.size() != 0 && nums[i] > nums[indicesQueue.back()]) {
                indicesQueue.pop_back();
            }
            indicesQueue.push_back(i);
            if (i >= k - 1) {
                result[i - k + 1] = nums[indicesQueue.front()];
            }
        }
        return result;
    }
};
// Time: O(n) amortized | Space: O(k)
```

Deque mein VALUES nahi, INDICES kyun store karte hain?

Index se hum jaan sakte hain ke element abhi window ke andar hai ya expire ho chuka (i-k+1 compare karke). Sirf value store karte toh expiry check karna possible hi nahi hota.

YAAD RAKHNE WALI CHEEZEIN

- Pattern: Monotonic Decreasing Deque — front hamesha current max
- Deque mein indices store karo, values nahi — expiry check ke liye zaroori
- Back se chote elements continuously pop karo — wo kabhi future max nahi ban sakte
- Front expire ho (front < i-k+1) toh pehle usse pop karo
- Har index ek hi baar push/pop hota hai — amortized O(n), naya pattern hai is guide mein

Part 2 Ka Pattern Summary

Part 2 mein 13 problems do bade patterns mein aate hain — Two Pointers aur Sliding Window.

PROBLEM	PATTERN	TIME	SPACE
#36 Valid Sudoku	HashSet Existence (3x)	O(1)	O(1)
#128 Longest Consecutive	HashSet Existence	O(n)	O(n)
#125 Valid Palindrome	Two Pointers Converge	O(n)	O(1)
#167 Two Sum II	Two Pointers on Sorted	O(n)	O(1)
#15 3Sum	Sort + Two Pointers	$O(n^2)$	O(1)
#11 Container With Most Water	Two Pointers Greedy	O(n)	O(1)
#42 Trapping Rain Water	Two Pointers Greedy	O(n)	O(1)
#121 Best Time Buy/Sell	Sliding Window (implicit)	O(n)	O(1)
#3 Longest Substring w/o Repeat	Sliding Window (variable, max)	O(n)	O(1)
#424 Longest Repeating Char	Sliding Window + Frequency	O(n)	O(1)
#567 Permutation in String	Sliding Window (fixed) + Frequency	O(n)	O(1)
#76 Minimum Window Substring	Sliding Window (variable, min)	O(n+m)	O(m)
#239 Sliding Window Maximum	Monotonic Deque	O(n)	O(k)

PART 3

Stack & Backtracking

Problems 20–26: Valid Parentheses se Largest Rectangle in Histogram tak. LIFO matching, custom stack design, postfix evaluation, monotonic stack, aur recursive construction — aur end mein saare 30 problems ka master pattern.

PROBLEM 20 · STACK

STACK

## 20 — Valid Parentheses

Problem Samajhte Hain

Ek string di gayi hai jisme sirf brackets '()[]{}' hain. Check karna hai ke ye valid hai ya nahi — matlab har opening bracket ka correct order mein matching closing bracket ho.

> **PROBLEM STATEMENT**
> Input: string s (sirf '()[]{}')   Output: true/false
> Example: "()[]{}" → true | "(]" → false

Core Intuition — Stack + Hash Map Matching

Stack LIFO (last-in-first-out) hai — bilkul bracket matching jaisa: sabse RECENT open bracket hi sabse pehle close hona chahiye. Ek hashmap banao jo closing bracket ko uske matching opening bracket se map kare. String traverse karo: closing bracket mile toh stack ke top se compare karo, opening bracket mile toh stack mein push karo.

```
class Solution {
public:
    bool isValid(string s) {
        unordered_map<char, char> PMap;
        PMap.insert({')', '('});
        PMap.insert({'}', '{'});
        PMap.insert({']', '['});
        stack<char> mStack;
        for (char c : s) {
            if (PMap.find(c) != PMap.end()) {
                if (!mStack.empty()) {
                    char topElement = mStack.top();
                    mStack.pop();
                    if (topElement != PMap[c]) { return false; }
                } else { return false; }
            } else { mStack.push(c); }
        }
        return mStack.empty();
    }
};
// Time: O(n) | Space: O(n)
```

> **COMMON MISTAKE**
> Stack empty hone par top() call karna undefined behavior hai — pehle !mStack.empty() check karna zaroori hai, jaisa code mein hai. Aur end mein mStack.empty() check bhoolna common bug hai — agar extra unmatched opening brackets bache reh gaye, string invalid honi chahiye.

YAAD RAKHNE WALI CHEEZEIN

- Pattern: Stack (LIFO) + Hash Map matching
- unordered_map closing→opening bracket map karta hai
- Stack empty check pehle karo, phir top()/pop()
- End mein stack empty honi chahiye — warna unclosed brackets bache reh gaye

PROBLEM 21 · STACK

STACK

## 155 — Min Stack

Problem Samajhte Hain

Ek stack design karo jisme push, pop, top, aur getMin — sab O(1) time mein ho. Normal stack mein min nikalna O(n) leta hai kyunki poora stack traverse karna padta hai.

> **PROBLEM STATEMENT**
> Design: push(x), pop(), top(), getMin() — sab O(1)
> Example: push(-2), push(0), push(-3) → getMin()=-3, pop(), top()=0, getMin()=-2

Core Intuition — Har Node Apna Min Yaad Rakhta Hai

Trick ye hai ke ek normal stack use karne ke bajaye, ek custom LINKED LIST banao jisme HAR node apna value AUR us waqt tak ka minimum dono store kare. Naya element push hone par uska min = min(nayi value, purane head ka min). Isse pop karne par purana min automatically wapas mil jata hai — kyunki wo pichle node mein already stored hai.

```
class Node {
public:
    int value; int min; Node* next;
    Node(int v, int m, Node* next) {
        value = v; min = m; this->next = next;
    }
};
class MinStack {
    Node* head;
public:
    MinStack() { head = NULL; }
    void push(int value) {
        if (head == NULL) { head = new Node(value, value, NULL); }
        else {
            int fMin = value < head->min ? value : head->min;
            head = new Node(value, fMin, head);
        }
    }
    void pop() { head = head->next; }
    int top() { return head->value; }
    int getMin() { return head->min; }
};
// Time: O(1) for all operations | Space: O(n)
```

> **INTERVIEW INSIGHT**
> Ye classic array-based min-stack (jisme do stacks ya {value,min} pairs store hote hain) ka linked-list variant hai. Dono approaches same idea use karti hain: HAR level par current minimum cache kar lo, taake pop hone par purana min automatically restore ho.

> **YAAD RAKHNE WALI CHEEZEIN**
> - Pattern: Stack jisme har element apna running min bhi carry kare
> - Linked list ka head hi stack ka top hai — pop matlab head = head->next
> - getMin() O(1) hai kyunki min already current node mein stored hai
> - Alternative: ek array-stack + parallel min-stack se bhi same effect milta hai

PROBLEM 22 · STACK

STACK

150 — Evaluate Reverse Polish Notation

Problem Samajhte Hain

Ek array of tokens diya hai jo Reverse Polish (postfix) notation mein ek arithmetic expression represent karta hai. Isse evaluate karke result nikalna hai.

> **PROBLEM STATEMENT**
> Input: vector<string> tokens (numbers aur +,-,*,/ operators)   Output: int — evaluated result
> Example: ["2","1","+","3","*"] → 9 ((2+1)*3)

Core Intuition — Postfix Ko Stack Se Evaluate Karo

Postfix notation ka fayda ye hai ke bracket ki zaroorat nahi hoti — bas ek stack maintain karo. Number mile toh push kar do. Operator mile toh stack se do numbers pop karo (pehla pop RIGHT operand hai, doosra pop LEFT operand), operation apply karo, result wapas push kar do. End mein stack mein sirf final answer bachta hai.

```
class Solution {
public:
    int evalRPN(vector<string>& tokens) {
        stack<int> mStack;
        int a = 0, b = 0, c = 0;
        for (string token : tokens) {
```

```
            if (isOperation(token)) {
                a = mStack.top(); mStack.pop();
                b = mStack.top(); mStack.pop();
                c = applyOperation(b, a, token);
                mStack.push(c);
            } else { mStack.push(stoi(token)); }
        }
        return mStack.top();
    }
    bool isOperation(string operation) {
        return operation == "+" || operation == "-" || operation == "*" || operation == "/";
    }
    int applyOperation(int a, int b, string operation) {
        if (operation == "+") return a + b;
        if (operation == "*") return a * b;
        if (operation == "-") return a - b;
        if (operation == "/") return a / b;
        return 0;
    }
};
// Time: O(n) | Space: O(n)
```

COMMON MISTAKE

Operand order ka khayal rakhna zaroori hai — subtraction aur division commutative nahi hote. Pehla pop() (a) hamesha RIGHT operand hai, doosra pop() (b) LEFT operand — operation hamesha applyOperation(b, a, token) honi chahiye, ulta karne se galat result.

YAAD RAKHNE WALI CHEEZEIN

- Pattern: Stack-based postfix evaluation
- Operand order: pehla pop = right, doosra pop = left
- stoi() se string token ko integer mein convert karo
- Postfix notation mein explicit brackets/precedence ki zaroorat nahi hoti

PROBLEM 23 · BACKTRACKING

BACKTRACKING

## 22 — Generate Parentheses

Problem Samajhte Hain

Ek number n diya hai. Saare combinations generate karne hain n pairs ke valid (well-formed) parentheses.

PROBLEM STATEMENT

Input: int n   Output: vector<string> — saare valid combinations
Example: n=3 → ["((()))","(()())","(())()","()(())","()()()"]

Core Intuition — Backtracking with Open/Close Counters

Recursive backtracking se construct karo: har step par ya toh '(' add karo (agar open count < n) ya ')' add karo (agar close count < open count — matlab abhi koi unmatched opening bracket bacha hai). Jab string ki length 2n ho jaye, ek valid combination mil gaya. Ye constraint (close < open) hi guarantee karta hai ke koi invalid combination kabhi banti hi nahi.

```
class Solution {
public:
    vector<string> generateParenthesis(int n) {
        vector<string> resultantParanthesis;
        backTrack(resultantParanthesis, "", 0, 0, n);
        return resultantParanthesis;
    }
    void backTrack(vector<string>& ansList, string constructedParanthesis,
                   int open, int close, int max) {
        if (constructedParanthesis.size() == max * 2) {
            ansList.push_back(constructedParanthesis);
            return;
        }
        if (open < max) {
            constructedParanthesis += "(";
            backTrack(ansList, constructedParanthesis, open + 1, close, max);
            constructedParanthesis = constructedParanthesis.substr(0, constructedParanthesis.length
        }
        if (close < open) {
            constructedParanthesis += ")";
            backTrack(ansList, constructedParanthesis, open, close + 1, max);
            constructedParanthesis = constructedParanthesis.substr(0, constructedParanthesis.length
        }
```

```
    }
};
// Time: O(4^n / √n) — Catalan number bound | Space: O(n) recursion depth
```

"Build, recurse, undo" pattern

Har recursive call ke baad constructedParanthesis.substr(...) se last character hata diya jata hai (undo step), taake agla branch (close bracket wala) fresh state se try ho sake. Ye backtracking ka signature move hai.

YAAD RAKHNE WALI CHEEZEIN

- Pattern: Backtracking — build, recurse, undo (substr se last char hatana)
- close < open constraint hi validity guarantee karta hai — extra check ki zaroorat nahi
- Base case: string length == max*2
- Ye is guide ka pehla pure recursion/backtracking based problem hai

PROBLEM 24 · STACK

STACK

## 739 — Daily Temperatures

Problem Samajhte Hain

Temperatures ka array diya hai. Har din ke liye dhundna hai ke kitne din baad temperature badhegi. Agar aage koi garam din nahi, toh 0.

PROBLEM STATEMENT

Input: vector<int> temperatures   Output: vector<int> — har index ke liye din jo wait karne padenge garam din ke liye

Example: temperatures=[73,74,75,71,69,72,76,73] → [1,1,4,2,1,1,0,0]

Core Intuition — Monotonic Decreasing Stack (Next Greater Element)

Stack mein {temperature, index} pairs store karo. Har naye din par, jab tak stack ke top ki temperature current se kam hai, usse pop karo aur us index ka answer = current_index - popped_index. Ye "next greater element" pattern hai — har element ke liye pehla bada element dhundna.

```
class Solution {
public:
    vector<int> dailyTemperatures(vector<int>& temperatures) {
        vector<int> result(temperatures.size(), 0);
        stack<pair<int, int>> mStack;
        for (int i = 0; i < temperatures.size(); i++) {
            int c = temperatures[i];
            while (mStack.size() != 0 && mStack.top().first < c) {
                int jumps = i - mStack.top().second;
                result[mStack.top().second] = jumps;
                mStack.pop();
            }
            pair<int, int> tempDetails = {c, i};
            mStack.push(tempDetails);
        }
        return result;
    }
};
// Time: O(n) amortized | Space: O(n)
```

INTERVIEW INSIGHT

Ye "Next Greater Element" (NGE) pattern ka classic variant hai. Monotonic decreasing stack har element ke liye pehla bada element track karta hai. Har element sirf ek baar push aur ek baar pop hota hai — isliye amortized O(n) time milta hai despite nested loop dikhne ke.

YAAD RAKHNE WALI CHEEZEIN

- Pattern: Monotonic Decreasing Stack — Next Greater Element
- Stack mein {value, index} store karo — index se distance calculate hota hai
- Current > top ? top ko pop karo aur answer set karo
- Jo elements bache reh gaye unka answer default 0 (koi garam din nahi aaya)
- Amortized O(n) — har element max 1 push + 1 pop

PROBLEM 25 · STACK

STACK

## 853 — Car Fleet

Problem Samajhte Hain

N cars same straight road par hain, har car apni position aur speed se target ki taraf ja rahi hai. Agar koi car peechay wali car ko catch kar le, toh dono ek "fleet" ban jati hain (speed slow wali ki ho jati hai). Kitni fleets target tak pahunchengi?

> **PROBLEM STATEMENT**
> Input: int target, vector<int> position, vector<int> speed   Output: int — number of car fleets
> Example: target=12, position=[10,8,0,5,3], speed=[2,4,1,1,3] → 3

Core Intuition — Sort by Position + Time Stack

Sabse pehle cars ko position ke hisaab se descending sort karo (sabse aage wali pehle). Har car ka "arrival time" = (target - position) / speed nikalo. Agar koi car apne se aage wali car se pehle (ya uske saath) pahunch sakti hai, toh wo fleet mein merge ho jayegi. Stack mein arrival times daalte jao — agar current time ≤ stack top, toh merge (pop nahi karte, bas skip). End mein stack size = fleets.

```
class Solution {
public:
    int carFleet(int target, vector<int>& position, vector<int>& speed) {
        vector<pair<int, int>> speedAndPositionPairs;
        int length = position.size();
        for (int i = 0; i < length; i++) {
            speedAndPositionPairs.push_back({position[i], speed[i]});
        }
        sort(speedAndPositionPairs.rbegin(), speedAndPositionPairs.rend());
        stack<double> timeStack;
        for (auto car : speedAndPositionPairs) {
            double arrivalTime = (target - car.first) / double(car.second);
            if (timeStack.empty() || arrivalTime > timeStack.top()) {
                timeStack.push(arrivalTime);
            }
        }
        return timeStack.size();
    }
};
// Time: O(n log n) for sort | Space: O(n)
```

> **INTERVIEW INSIGHT**
> Is problem ko stack ke bina bhi solve kar sakte hain — sirf ek variable mein last fleet time track karo. Lekin stack conceptually clear hai: har fleet ek "layer" represent karta hai. Descending sort isliye kyunki hum peechay se dekhna chahte hain — kya peechay wali car aage wali ko catch kar sakti hai?

> **YAAD RAKHNE WALI CHEEZEIN**
> - Pattern: Sort + Monotonic Stack (arrival times)
> - Descending sort by position — peechay se front ki taraf analyze karo
> - Arrival time = (target - position) / speed
> - Current time > stack top ? New fleet : Merge into existing fleet
> - Stack size hi final answer hai

PROBLEM 26 · STACK

STACK

## 84 — Largest Rectangle in Histogram

Problem Samajhte Hain

Heights ka array diya hai jo histogram ki bars represent karta hai. Sabse bada rectangle dhundna hai jo histogram ke andar ban sakta hai.

> **PROBLEM STATEMENT**
> Input: vector<int> heights   Output: int — maximum rectangle area
> Example: heights=[2,1,5,6,2,3] → 10

Core Intuition — Monotonic Increasing Stack + Width Calculation

Stack mein {start_index, height} pairs store karo. Jab current height stack top se choti ho, matlab previous bar ab extend nahi kar sakta. Usse pop karo aur area calculate karo: height * (current_index - start_index). Jo bhi bars bache unka start index update karo. End mein jo stack mein bache hain unka width = array_end - start_index.

```
class Solution {
public:
    int largestRectangleArea(vector<int>& heights) {
        stack<pair<int, int>> mStack;
        int length = heights.size();
        int maxArea = 0;
        for (int i = 0; i < length; i++) {
            int h = heights[i];
            int start = i;
            while (mStack.size() > 0 && mStack.top().second > h) {
                int index = mStack.top().first;
                int height = mStack.top().second;
                mStack.pop();
                maxArea = max(maxArea, height * (i - index));
                start = index;
            }
            mStack.push({start, h});
        }
        while (mStack.size() > 0) {
            int index = mStack.top().first;
            int h = mStack.top().second;
            mStack.pop();
            maxArea = max(maxArea, h * (length - index));
        }
        return maxArea;
    }
};
// Time: O(n) amortized | Space: O(n)
```

INTERVIEW INSIGHT

Ye LeetCode ka sabse mashhoor "hard" stack problem hai. Key insight: har bar ka rectangle uske dono sides tak extend hota hai jahan tak usse choti bars na milen. Stack har bar ka "left boundary" (start index) store karta hai. Jab choti bar aati hai, previous bar ka final right boundary mil jata hai.

YAAD RAKHNE WALI CHEEZEIN

- Pattern: Monotonic Increasing Stack — har bar ka left boundary track karna
- Stack mein {start_index, height} — start index se width calculate hoti hai
- Current < top ? top ko pop karo aur area = height * (i - start_index)
- Start = popped index — current bar utni peechay se extend kar sakti hai
- End mein bache hue bars ka width = length - start_index
- Amortized O(n) — har element max 1 push + 1 pop

Part 3 Ka Pattern Summary

Part 3 stack aur backtracking ka naya territory hai — pehli baar is guide mein LIFO order aur recursive construction use ho rahe hain. Ab 3 naye monotonic stack problems bhi shamil hain.

PROBLEM	PATTERN	TIME	SPACE
#20 Valid Parentheses	Stack + HashMap Matching	O(n)	O(n)
#155 Min Stack	Stack w/ Embedded Running Min	O(1) all ops	O(n)
#150 Evaluate RPN	Stack-based Postfix Eval	O(n)	O(n)
#22 Generate Parentheses	Backtracking	O(4^n/√n)	O(n)
#739 Daily Temperatures	Monotonic Decreasing Stack (NGE)	O(n)	O(n)
#853 Car Fleet	Sort + Monotonic Stack	O(n log n)	O(n)
#84 Largest Rectangle	Monotonic Increasing Stack	O(n)	O(n)

DEKHO KAISE YE CHAIN BANTI HAI

Valid Parentheses (#20) ka LIFO matching seedha Min Stack (#155) aur Evaluate RPN (#150) mein reuse hota hai. Generate Parentheses (#22) isi bracket-validity ko REVERSE karta hai — check karne ke bajaye khud construct karta hai. Daily Temperatures (#739) monotonic stack pattern introduce karta hai jo Car Fleet (#853) aur Largest Rectangle (#84) mein advanced form mein reuse hota hai.

PART 4

Binary Search

Problems 27–30: Binary Search se Search in Rotated Sorted Array tak. Sorted space ko half karna, search space shrink karna, aur rotated array mein pivot dhundna — sab Roman Urdu mein.

PROBLEM 27 · BINARY SEARCH

BINARY SEARCH

## 704 — Binary Search

Problem Samajhte Hain

Ek sorted integer array aur target diya hai. Target ka index dhundna hai. Agar nahi milta, -1 return karna hai. Classic binary search — array ko baar baar half karte jao.

> **PROBLEM STATEMENT**
> Input: vector<int> nums (sorted ascending), int target   Output: int — target ka index ya -1
> Example: nums=[-1,0,3,5,9,12], target=9 → 4

Core Intuition — Search Space Ko Half Karo

Sorted array ka faida: mid nikalo, agar target mid se bada hai toh left half discard karo, agar chota hai toh right half discard karo. Har step mein search space half ho jata hai — O(log n) time.

```
class Solution {
public:
    int search(vector<int>& nums, int target) {
        int left = 0, right = nums.size() - 1;
        while (left <= right) {
            int mid = left + (right - left) / 2;
            if (target > nums[mid]) {
                left = mid + 1;
            } else if (target < nums[mid]) {
                right = mid - 1;
            } else {
                return mid;
            }
        }
        return -1;
    }
};
// Time: O(log n) | Space: O(1)
```

> **INTERVIEW INSIGHT**
> mid = (left + right) / 2 ki jagah mid = left + (right - left) / 2 use karo — integer overflow avoid karne ke liye. while(left <= right) vs while(left < right) mein farq samajhna zaroori hai: <= use karte hain jab exact match dhundna ho, < use karte hain jab boundary dhundni ho.

YAAD RAKHNE WALI CHEEZEIN

- Pattern: Binary Search — sorted array mein search space half karna
- mid = left + (right - left) / 2 — overflow safe
- left <= right loop — exact index dhundne ke liye
- O(log n) time — brute force O(n) se exponentially behtar
- Ye base pattern hai jo sab rotated array problems mein reuse hota hai

PROBLEM 28 · BINARY SEARCH

BINARY SEARCH

## 875 — Koko Eating Bananas

Problem Samajhte Hain

Koko ko k piles of bananas hain. Wo har hour mein k bananas kha sakti hai. Agar pile mein k se kam hain, toh wo pile khatam karke hour waste nahi karti — agle pile pe move kar jati hai. H hours mein saare bananas khatam karne ke liye minimum k dhundna hai.

> **PROBLEM STATEMENT**
> Input: vector<int> piles, int h   Output: int — minimum integer k such that Koko can eat all bananas in h hours
> Example: piles=[3,6,7,11], h=8 → 4

Core Intuition — Binary Search on Answer

Answer (k) ka range fixed hai: 1 se leke max(piles) tak. Har possible k ke liye check kar sakte hain ke kitne hours lagenge — ye O(n) check hai. Ab is range pe binary search lagao: agar current k se hours ≤ h hain, toh aur chota k try karo (right = k-1), warna bada k chahiye (left = k+1).

```
class Solution {
public:
    int minEatingSpeed(vector<int>& piles, int h) {
        int k = 0;
        int l = 1, r = *max_element(piles.begin(), piles.end());
        int res = r;
        while (l <= r) {
            k = (l + r) / 2;
            double total = 0;
            for (int p : piles) {
                total += ceil(p / double(k));
            }
            if (total <= h) {
                res = min(res, k);
                r = k - 1;
            } else {
                l = k + 1;
            }
        }
        return res;
    }
};
// Time: O(n log m) where m = max(piles) | Space: O(1)
```

INTERVIEW INSIGHT

"Binary Search on Answer" pattern — jab direct answer nahi pata lekin answer ka range pata ho, aur har possible answer ko validate karne ka efficient way ho. Ye pattern bahut common hai: minimum capacity, minimum speed, maximum sweetness, etc.

YAAD RAKHNE WALI CHEEZEIN

- Pattern: Binary Search on Answer — answer ka range pe binary search
- Range: 1 to max(piles) — k itna bada hona chahiye ke sabse badi pile ek hour mein khatam ho
- Validation function: har pile ke liye ceil(pile/k) sum karo
- total <= h ? right shrink karo (aur chota k dhundo) : left badhao
- res variable minimum valid k track karta hai

PROBLEM 29 · BINARY SEARCH

BINARY SEARCH

153 — Find Minimum in Rotated Sorted Array

Problem Samajhte Hain

Ek sorted array ko kisi unknown pivot se rotate kiya gaya hai. Array mein minimum element dhundna hai. Array mein duplicates nahi hain.

PROBLEM STATEMENT

Input: vector<int> nums (rotated sorted, unique)   Output: int — minimum element
Example: nums=[3,4,5,1,2] → 1 | nums=[4,5,6,7,0,1,2] → 0

Core Intuition — Sorted Half Discard Karo

Rotated sorted array mein hamesha ek half sorted hota hai. Agar nums[mid] >= nums[left], matlab left half sorted hai — minimum right half mein hoga. Warna right half sorted hai — minimum left half mein hoga (ya khud mid hi minimum ho).

```
class Solution {
public:
    int findMin(vector<int>& nums) {
        int l = 0, r = nums.size() - 1;
        int res = nums[l];
        int m = 0;
        while (l <= r) {
            if (nums[l] < nums[r]) {
                res = min(res, nums[l]);
                break;
            }
            m = (l + r) / 2;
            res = min(res, nums[m]);
            if (nums[m] >= nums[l]) {
                l = m + 1;
            } else {
                r = m - 1;
```

```
            }
        }
        return res;
    }
};
// Time: O(log n) | Space: O(1)
```

INTERVIEW INSIGHT

nums[l] < nums[r] check early exit ke liye zaroori hai — agar left < right hai, matlab ye portion already sorted hai, toh minimum nums[l] hi hoga. Ye optimization bina bhi kaam karegi lekin average case behtar banati hai.

YAAD RAKHNE WALI CHEEZEIN

- Pattern: Modified Binary Search on Rotated Array
- Hamesha ek half sorted hota hai — usse minimum reject karo
- nums[mid] >= nums[left] ? Left sorted → min right mein : Right sorted → min left mein
- res = min(res, nums[m]) har iteration mein — mid hi minimum ho sakta hai
- Duplicates nahi hain isliye >= comparison safe hai

PROBLEM 30 · BINARY SEARCH

BINARY SEARCH

33 — Search in Rotated Sorted Array

Problem Samajhte Hain

Rotated sorted array mein target element ka index dhundna hai. Agar nahi milta, -1 return karna hai. Ye #153 ka extension hai — ab sirf minimum nahi, koi bhi element dhundna hai.

PROBLEM STATEMENT

Input: vector<int> nums (rotated sorted), int target   Output: int — target ka index ya -1
Example: nums=[4,5,6,7,0,1,2], target=0 → 4

Core Intuition — Pehle Decide Karo Kaunsa Half Sorted Hai

Har mid par pehle check karo ke left half sorted hai ya right half. Phir target us sorted half mein lie karta hai ya nahi — accordingly move karo. Agar left sorted hai aur target left sorted range mein hai, left mein jao. Warna right mein jao.

```
class Solution {
public:
    int search(vector<int>& nums, int target) {
        int l = 0, r = nums.size() - 1;
        int m = 0;
        while (l <= r) {
            m = (l + r) / 2;
            if (nums[m] == target) { return m; }
            // Left Sorted Portion
            if (nums[m] >= nums[l]) {
                if (target > nums[m] || target < nums[l])
                    l = m + 1;
                else
                    r = m - 1;
            }
            // Right Sorted Portion
            else {
                if (target < nums[m] || target > nums[r]) {
                    r = m - 1;
                } else {
                    l = m + 1;
                }
            }
        }
        return -1;
    }
};
// Time: O(log n) | Space: O(1)
```

INTERVIEW INSIGHT

Ye problem interviewers ka favorite hai kyunki standard binary search ko modify karna padta hai. Key: pehle sorted half identify karo, phir target usmein fit hota hai ya nahi. nums[m] >= nums[l] se left sorted pata chalta hai — equal sign isliye kyunki l==m ho sakta hai.

YAAD RAKHNE WALI CHEEZEIN

- Pattern: Modified Binary Search on Rotated Array — Target Search
- Pehle decide: left sorted hai ya right sorted?
- Left sorted (nums[m] >= nums[l]): target [l,m] mein hai ? r = m-1 : l = m+1
- Right sorted: target [m,r] mein hai ? l = m+1 : r = m-1
- Boundary conditions carefully handle karo — >= vs > mein farq
- Ye #153 (Find Minimum) ka direct extension hai

Part 4 Ka Pattern Summary

Part 4 mein binary search ke 4 core variants cover hote hain — standard search, search on answer, aur rotated array modifications.

PROBLEM	PATTERN	TIME	SPACE
#704 Binary Search	Standard Binary Search	O(log n)	O(1)
#875 Koko Eating Bananas	Binary Search on Answer	O(n log m)	O(1)
#153 Find Minimum (Rotated)	Modified BS — Rotated Array	O(log n)	O(1)
#33 Search (Rotated)	Modified BS — Rotated Target	O(log n)	O(1)

DEKHO KAISE YE CHAIN BANTI HAI

Standard Binary Search (#704) base hai. Usi concept ko "Binary Search on Answer" (#875) mein apply karte hain jahan answer ka range pata ho. Rotated array problems (#153, #33) mein same binary search ko modify karte hain — pehle sorted half identify karo, phir decision lo. #153 sirf minimum dhundta hai, #33 koi bhi target dhundta hai using same sorted-half logic.

MASTER PATTERN MAP

Sab Problems Ka Connection

30 problems, 7 core patterns — dekhte hain kaise sab aapas mein juray hue hain.

Pattern 1 — HashSet Existence Check — O(1) lookup

Idea: "Ye element pehle dekha hai kya?" — sirf existence check karni hai, frequency nahi. unordered_set use hota hai.

PROBLEM	KAISE USE HOTA HAI
#217 Contains Duplicate	Seen elements ka set — duplicate mila toh true
#128 Longest Consecutive Sequence	Set se sequence-start check: (n-1) set mein hai ya nahi
#36 Valid Sudoku	Row/Col/Box ke liye teen parallel sets — duplicate digit check
#3 Longest Substring w/o Repeat	Current window ke unique characters ka set

Pattern 2 — Frequency Array / Map

Idea: "Kitni baar aaya hai?" — count matter karta hai, sirf presence nahi. Fixed alphabet (26 letters) ho toh int arr[26] hamesha unordered_map se fast hai.

PROBLEM	KAISE USE HOTA HAI
#242 Valid Anagram	int arr[26] — dono strings ki frequency compare
#49 Group Anagrams	Frequency-encoded string as hashmap key
#347 Top K Frequent	unordered_map frequency + min-heap
#424 Longest Repeating Char Replacement	freq[26] sliding window ke andar — maxFreq track
#567 Permutation in String	Do freq[26] arrays match — fixed-size window slide

#76 Minimum Window Substring	tMap/sMap frequency + required/create counters

> **CHAIN:** Valid Anagram (#242) mein sikha frequency array trick seedha Permutation in String (#567) mein reuse hota hai — bas ab ye ek FIXED-SIZE SLIDING WINDOW ke andar chal raha hai. Aur Longest Repeating Char Replacement (#424) usi trick ko VARIABLE window ke andar use karta hai.

Pattern 3 — Two Pointers Converge

Do pointers opposite ends se shuru hoke ek dusre ki taraf move karte hain. Kaam karta hai jab data sorted ho ya symmetric decision lene ka koi monotonic rule ho.

PROBLEM	CONVERGENCE RULE
#125 Valid Palindrome	Characters match karo, mismatch pe false
#167 Two Sum II	sum > target: right--, sum < target: left++
#15 3Sum	Two Sum II ka extension — outer loop + inner two-pointer
#11 Container With Most Water	Choti height wala pointer move karo (greedy)
#42 Trapping Rain Water	Choti height wala pointer + running max

Pattern 4 — Sliding Window Expand-Shrink

Right pointer window expand karta hai, kisi condition ke violate hone par left pointer shrink karta hai. Fixed-size aur variable-size dono variants hote hain.

PROBLEM	WINDOW TYPE	SHRINK CONDITION
#121 Best Time to Buy/Sell Stock	Implicit (running min)	Naya price purane min se kam ho
#3 Longest Substring w/o Repeat	Variable (maximize)	Character repeat ho jaye
#424 Longest Repeating Char Replacement	Variable (maximize)	window - maxFreq > k
#567 Permutation in String	Fixed (= s1.length())	Har step slide — shrink hamesha hoti hai
#76 Minimum Window Substring	Variable (minimize)	create == required tak shrink karo
#239 Sliding Window Maximum	Fixed (=k), Monotonic Deque	Back se chote elements pop, front expire hone par pop

> **CHAIN:** Longest Substring (#3) MAXIMIZE karta hai window jab tak condition breaks. Minimum Window Substring (#76) iska "mirror image" hai — MINIMIZE karta hai window jab tak condition valid rehti hai.

Pattern 5 — Stack (LIFO Matching & Tracking)

Idea: "Sabse recent cheez sabse pehle nikalti hai." Bracket matching, running-min tracking, postfix evaluation, aur monotonic stack — sab mein LIFO order hi kaam karta hai.

PROBLEM	KAISE USE HOTA HAI
#20 Valid Parentheses	Closing bracket mile toh stack top se match, warna push
#155 Min Stack	Har node apna value + us waqt tak ka min store karta hai
#150 Evaluate RPN	Operands push hote hain, operator mile toh do pop karke result push
#739 Daily Temperatures	Monotonic decreasing stack — Next Greater Element
#853 Car Fleet	Monotonic stack — arrival times track karna
#84 Largest Rectangle	Monotonic increasing stack — left boundaries track karna

> **CHAIN:** Valid Parentheses (#20) ka LIFO matching seedha Min Stack (#155) aur Evaluate RPN (#150) mein reuse hota hai. Daily Temperatures (#739) monotonic stack introduce karta hai jo Car Fleet (#853) aur Largest Rectangle (#84) mein advanced form mein reuse hota hai.

Pattern 6 — Backtracking (Build → Recurse → Undo)

Idea: Ek choice karo, recurse karo, phir undo karke agla choice try karo. Constraints invalid paths ko shuru mein hi rok dete hain.

PROBLEM	CONSTRUCTION RULE
#22 Generate Parentheses	open < n toh '(' add karo, close < open toh ')' add karo — phir undo karke agla branch try karo

> **CHAIN:** Valid Parentheses (#20) validity CHECK karta hai stack se. Generate Parentheses (#22) validity ki GUARANTEE deta hai construction rules se — dono ek hi bracket-matching concept ke do opposite directions hain: ek verify karta hai, doosra construct karta hai.

Pattern 7 — Binary Search (Search Space Halving)

Idea: Sorted ya ordered search space ko half karte jao. Standard form, answer-space form, aur rotated array modifications — teeno variants.

PROBLEM	VARIANT	KEY IDEA
#704 Binary Search	Standard	mid nikalo, half discard karo
#875 Koko Eating Bananas	Search on Answer	Range [1, max] pe BS, validate with O(n) check
#153 Find Minimum (Rotated)	Rotated Array	Sorted half identify karo, min usmein nahi hoga
#33 Search (Rotated)	Rotated Array + Target	Sorted half identify karo, target usmein hai ya nahi

> **CHAIN:** Standard BS (#704) base hai. Same concept "Binary Search on Answer" (#875) mein apply hota hai. Rotated array problems (#153, #33) mein standard BS modify karte hain — pehle sorted half identify karo. #153 minimum dhundta hai, #33 koi bhi target dhundta hai using same sorted-half logic.

Complete Problem-to-Pattern Table (30 Problems)

#	PROBLEM	PRIMARY PATTERN	TIME	SPACE
217	Contains Duplicate	HashSet Existence	O(n)	O(n)
242	Valid Anagram	Frequency Array	O(n)	O(1)
1	Two Sum	HashMap + Complement	O(n)	O(n)
49	Group Anagrams	Frequency Array (as key)	O(nk)	O(nk)
347	Top K Frequent	Frequency + Min-Heap	O(n log k)	O(n)
238	Product Except Self	Prefix x Postfix	O(n)	O(1)
36	Valid Sudoku	HashSet Existence (3x)	O(1)	O(1)
128	Longest Consecutive	HashSet Existence	O(n)	O(n)
125	Valid Palindrome	Two Pointers	O(n)	O(1)
167	Two Sum II	Two Pointers	O(n)	O(1)
15	3Sum	Sort + Two Pointers	$O(n^2)$	O(1)
11	Container With Most Water	Two Pointers Greedy	O(n)	O(1)
42	Trapping Rain Water	Two Pointers Greedy	O(n)	O(1)
121	Best Time to Buy/Sell Stock	Sliding Window (implicit)	O(n)	O(1)

3	Longest Substring w/o Repeat	Sliding Window (variable, max)	O(n)	O(1)
424	Longest Repeating Char Replacement	Sliding Window + Frequency	O(n)	O(1)
567	Permutation in String	Sliding Window (fixed) + Frequency	O(n)	O(1)
76	Minimum Window Substring	Sliding Window (variable, min)	O(n+m)	O(m)
239	Sliding Window Maximum	Monotonic Deque	O(n)	O(k)
20	Valid Parentheses	Stack + HashMap Matching	O(n)	O(n)
155	Min Stack	Stack w/ Embedded Running Min	O(1) all ops	O(n)
150	Evaluate Reverse Polish Notation	Stack-based Postfix Eval	O(n)	O(n)
22	Generate Parentheses	Backtracking	O(4^n/√n)	O(n)
739	Daily Temperatures	Monotonic Decreasing Stack (NGE)	O(n)	O(n)
853	Car Fleet	Sort + Monotonic Stack	O(n log n)	O(n)
84	Largest Rectangle in Histogram	Monotonic Increasing Stack	O(n)	O(n)
704	Binary Search	Standard Binary Search	O(log n)	O(1)
875	Koko Eating Bananas	Binary Search on Answer	O(n log m)	O(1)
153	Find Minimum in Rotated Sorted Array	Modified BS — Rotated Array	O(log n)	O(1)
33	Search in Rotated Sorted Array	Modified BS — Rotated Target	O(log n)	O(1)

Cross-Pattern Connections — Sabse Interesting Overlaps

- **Frequency array + Sliding Window** = Longest Repeating Char Replacement aur Permutation in String dono, "frequency array" (Pattern 2) ko "sliding window" (Pattern 4) ke andar embed karte hain.
- **Two Sum family:** Two Sum (#1, hashmap, unsorted) aur Two Sum II (#167, two-pointer, sorted) same problem hain — data ka sorted state hi decide karta hai kaunsi technique behtar hai.
- **Greedy "move the smaller side":** Container With Water aur Trapping Rain Water dono is proof ko share karte hain.
- **Existence vs Frequency:** Contains Duplicate (existence) aur Valid Anagram (frequency) dikhte similar hain lekin fundamentally different hain.
- **Check vs Generate:** Valid Parentheses (check) aur Generate Parentheses (construct) same bracket-matching rule ko do opposite directions mein use karte hain.
- **Monotonic Stack family:** Daily Temperatures (decreasing, NGE), Car Fleet (sort + decreasing), Largest Rectangle (increasing, left boundaries) — teeno monotonic stack ke variants hain.
- **Binary Search family:** Standard (#704) → Search on Answer (#875) → Rotated Array (#153) → Rotated Target (#33) — difficulty gradually badhti hai, base concept wahi rehta hai.

FINAL INTERVIEW MINDSET

SCENARIO	PATTERN / DATA STRUCTURE
Sirf "existence" chahiye	unordered_set (Pattern 1)
"Count/frequency" chahiye + fixed alphabet	int arr[26] (Pattern 2)
Data sorted hai ya symmetric decision hai	Two Pointers (Pattern 3)
Substring/subarray ka best size chahiye	Sliding Window (Pattern 4)
Maximize karna hai	window invalid hone par shrink
Minimize karna hai	window valid hone par shrink
Nested/matching pairs ya LIFO order chahiye	Stack (Pattern 5)
Next greater/smaller element dhundna hai	Monotonic Stack (Pattern 5 extension)
Rules ke saath combinations construct karni hain	Backtracking (Pattern 6)
Sorted array mein kuch dhundna hai	Binary Search (Pattern 7)

Answer ka range pata ho, validate kar sakte ho	Binary Search on Answer (Pattern 7)
Rotated sorted array mein dhundna hai	Modified Binary Search (Pattern 7)

> **30 problems sirf 7 patterns hain** — naya problem dekho toh sabse pehle poocho: "ye kaunsa pattern hai?"

STL CHEAT SHEET

Data Structures Ka Naqsha

In 30 problems mein use hui har C++ STL data structure — kya hai, kaise kaam karti hai, aur kahan use hui.

Sequence Container

vector<T> — Dynamic Array

Resizable array jo memory mein contiguous hoti hai — isliye index se access CPU cache-friendly aur O(1) hai. Guide ke bilkul har problem ka base data structure yahi hai.

OPERATION	COMPLEXITY	NOTE
v[i] / v.at(i)	O(1)	at() bounds check karta hai, [] nahi
v.push_back(x)	O(1) amortized	Kabhi kabhi resize hone par O(n)
v.size()	O(1)	
v.reserve(n)	O(1) amortized	Pehle se memory le kar resizing avoid karta hai
sort(v.begin(), v.end())	O(n log n)	Introsort — 3Sum jaisi problems mein pehla step

deque<T> — Double-Ended Queue

Dono ends (front aur back) se O(1) mein push/pop karne deta hai, saath hi random access bhi O(1) hai. Jab kisi sliding window ke andar ek "monotonic" order maintain karna ho, deque hi sahi tool hai.

OPERATION	COMPLEXITY
dq.push_back(x) / push_front(x)	O(1)
dq.front() / dq.back()	O(1)
dq.pop_front() / pop_back()	O(1)

Hash Container

unordered_set<T> — Hash Set

Unique elements ka collection, koi order guaranteed nahi. Internally hash table hai isliye insert/lookup average O(1) hote hain — "ye element dekha hai kya" wale existence-check problems ke liye perfect.

OPERATION	COMPLEXITY	NOTE
s.insert(x)	O(1) average	Worst case O(n) agar collisions zyada hon
s.count(x) / s.find(x)	O(1) average	count() 0 ya 1 return karta hai
s.erase(x)	O(1) average	
s.reserve(n)	O(1) amortized	Rehashing avoid karta hai — bade inputs par zaroori

unordered_map<K,V> — Hash Map

Key-value pairs store karta hai, hash table based, average O(1) operations. Jab "kitni baar aaya" ya "kis key se kya associated hai" jaanna ho, tab ye default choice hai.

OPERATION	COMPLEXITY	NOTE
m[key]	O(1) average	Key na ho toh default value CREATE kar deta hai
m.insert({k,v})	O(1) average	Existing key ko UPDATE nahi karta — common mistake

m.count(key) / m.find(key)	O(1) average	Existence check ke liye insert() se safe
m.size()	O(1)	Group Anagrams mein groups ki count ke liye

Fixed Array (Non-STL)

int arr[26] — Fixed Frequency Array

Jab alphabet fixed size ka ho (26 lowercase letters), ek plain C-style stack array unordered_map se hamesha fast hota hai — no hashing overhead, cache-friendly, aur guaranteed O(1) access.

OPERATION	COMPLEXITY	NOTE
arr[c-'a']++	O(1)	Character ko 0-25 index mein convert karta hai
Compare do arrays	O(26) = O(1)	Fixed size hone ki wajah se constant

Container Adapter

priority_queue<T> — Heap

Binary heap par based container adapter — default MAX-heap (sabse bada element top par). greater<T> comparator do toh MIN-heap ban jata hai. "Top k" ya "smallest/largest so far" wale problems ke liye ideal.

OPERATION	COMPLEXITY	NOTE
pq.push(x)	O(log n)	
pq.pop()	O(log n)	Void return karta hai — pehle top() lo, phir pop()
pq.top()	O(1)	Current max (ya min-heap mein min)

stack<T> — LIFO Stack

Container adapter jo Last-In-First-Out order follow karta hai (internally deque par based). Sirf ek end ("top") se push/pop hota hai — bracket matching, nested structures, aur postfix evaluation jaise "sabse recent pehle nikalta hai" wale problems ke liye default tool.

OPERATION	COMPLEXITY	NOTE
st.push(x)	O(1)	
st.pop()	O(1)	Void return karta hai — pehle top() lo, phir pop()
st.top()	O(1)	Sabse upar wala (most recently pushed) element
st.empty()	O(1)	top()/pop() se pehle hamesha check karo

Utility / Algorithms

pair<A,B>, string, aur <algorithm>

pair chota do-value container hai (.first, .second) — heap mein {freq, num} store karne jaisi jagah useful. string apni khud ki class hai (.substr, .at, += sab O(1)-ish amortized). <algorithm> header se sort(), max(), min() jaise functions milte hain.

FUNCTION	COMPLEXITY	NOTE
s.substr(pos, len)	O(len)	Minimum Window Substring mein final answer nikalne ke liye
isalnum(c) / tolower(c)	O(1)	ctype.h se, Valid Palindrome mein
sort(v.begin(), v.end())	O(n log n)	3Sum mein sabse pehla step
max(a,b) / min(a,b)	O(1)	Har doosri problem mein running max/min ke liye
ceil(x)	O(1)	Koko Eating Bananas mein hours calculate karne ke liye
stoi(s)	O(n)	String token ko integer mein convert — Evaluate RPN mein

Quick Decision Guide

ZAROORAT	DATA STRUCTURE
Sirf existence check karni hai	unordered_set
Key se value map karni hai (unsorted)	unordered_map
Fixed alphabet ki frequency ginni hai	int arr[26]
Top-k ya running min/max chahiye	priority_queue
Sliding window ka running max/min chahiye	deque (monotonic)
Sorted data pe do pointers chalane hain	vector + sort()
Nested/matching pairs ya LIFO order chahiye	stack
Next greater/smaller element chahiye	stack (monotonic)
Rules ke saath combinations construct karni hain	recursion + backtracking
Sorted/rotated array mein kuch dhundna hai	binary search on vector

---

NeetCode 150 — Complete

30 Problems · 7 Core Patterns · Roman Urdu

SYED ABDUL ALI SHAH · DSA PREP

"Naya problem dekho toh sabse pehle poocho: ye kaunsa pattern hai?"